Control Structures

L19 - Selection Statements- switch statement

Department of Computer Science
University of Pretoria

- Used for multiple choices to be made based on the value of a variable.
- This is handy to use instead of multiple nested if-statements or chained else-if statements.
- It is useful when you have a **variable that can be one of several discrete values**, and the action that you need to perform if based on the value.

- switch structure is an alternate to if-else
- switch (integral) expression is evaluated first
- Value of the expression determines which corresponding action is taken
- Expression is sometimes called the selector

The syntax of a **switch**–statement in C++ is as follows:

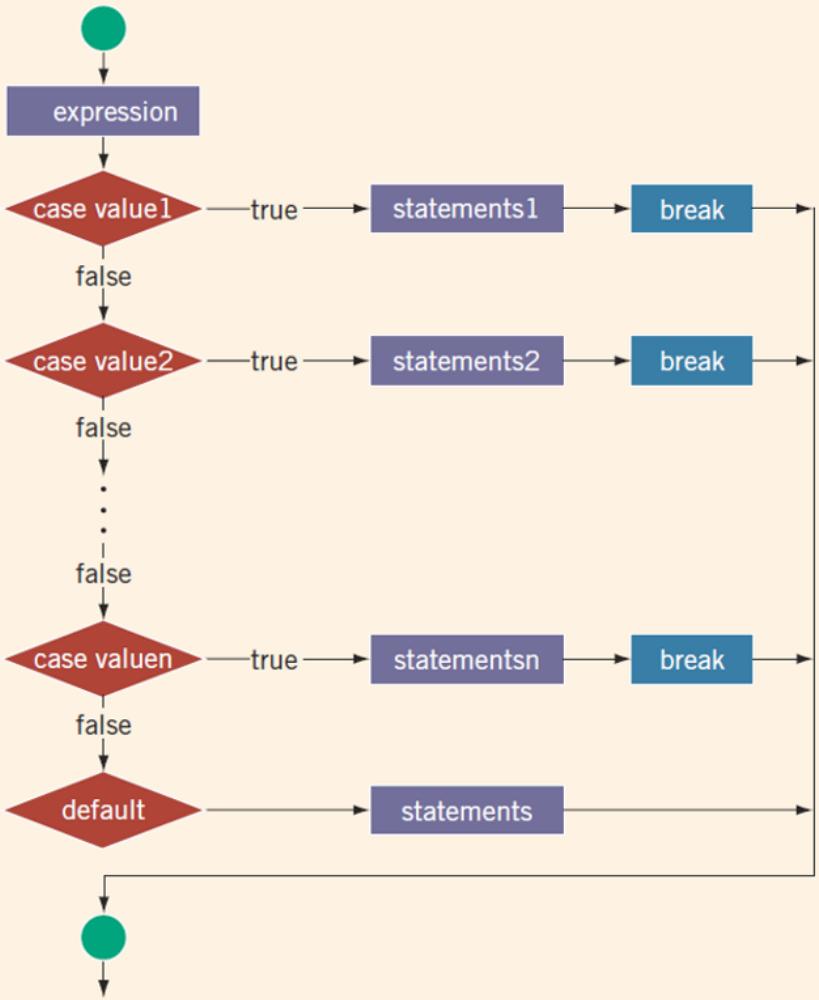
```
switch (expression)
{
case value1:
     statement1;
     break;
case value2:
     statement2;
     break;
  . . .

default:
     statement2;
}
```

- switch(expression): Evaluates an expression of integral type (int, char, etc.)
- case value1:: Matches the expression value with value1. Code block to execute if there's a match.
- break;: Exits the switch block after a matching case is found (optional but highly recommended).

- default:: Code block to execute if no matching case is found.
- One or more statements may follow a case label
- Braces are not needed to turn multiple statements into a single compound statement

- The break statement may or may not appear after each statement
- switch, case, break, and default are reserved words


expression
case value1
true
statements1
break
false
case value2
true
statements2
break
false
false
case valuen
true
statementsn
break
false
default
statements

```
1  switch (expression)
2  {
3  case value1:
4       statement1;
5       statement2;
6       .
7       .
8       break;
9  case value2:
10      statementm;
11      statementn;
12      .
13      .
14      break;
15   . . .
16 default:
17      statementaa;
18      statementbb;
19      .
20      .
21 }
```

Exercise Solution One:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int option = 2;
5      switch(option) {
6      case 1:
7          cout << "Option-1";
8          break;
9      case 2:
10          cout<<"Option-2";
11     case 3:
12         cout << "Option-3";
13         break;
14     default:
15         cout << "Invalid-option";
16     }
17     return 0;
18 }
```

Exercise Solution Two:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int option = 2;
5      switch(option) {
6      case 1:
7          cout << "Option-1";
8          break;
9      case 2:
10     case 3:
11         cout << "Option-2-or-3";
12         break;
13     default:
14         cout << "Invalid-option";
15     }
16     return 0;
17 }
```

- Example One solution:
  OUTPUT: Option 2 Option 3
- Example Two solution:
  OUTPUT: Option 2 or 3

In this example, the expression in the switch is a variable identifier. variable grade is of type char, which is a integral type.
The possible values of grade are 'A', 'B','C','D', and 'F'.
Each case label specifies a different action to take,depending on the value of grade.

```
#include <iostream>
using namespace std;
int main() {
    char grade = 'A';
    switch(grade) {
    case 'A':
        cout << "The grade is 4.0.";
        break;
    case 'B':
        cout << "The grade is 3.0.";
        break;
     case 'C':
        cout << "The grade is 2.0.";
        break;
```

```
case 'D':
        cout << "The grade is 1.0.";
        break;
case 'F':
        cout << "The grade is 0.0.";
        break;
    default:
        cout << "The grade is invalid";
    }
    return 0;
}
```

If the value of grade is 'A', the output is "The grade is 4.0".

If the value of grade is 'F', the output is "The grade is 0.0".

If the value of grade is 'E', the output is "The grade is invalid".

In the switch with multiple case labels, Execution "falls thru" until break –switch provides a "point of entry":

```
1  switch (expression)
2  {
3  case "A":
4  case "a"
5       statement1;
6       statement2;
7       break;
8  case "B"
9  case 'b'
10      statementm;
11      statementn;
12      break;
13 default:
14      statementaa;
15      statementbb;
16 }
```

Example 1

```
#include <iostream>
using namespace std;
int main() {
char grade = 'c';
     switch(grade) {
     case 'A':
     case 'a':
          cout << "The grade is 4.0.";
          break;
     case 'B':
     case 'b':
          cout << "The grade is 3.0.";
          break;
      case 'C':
      case 'c':
          cout << "The grade is 2.0.";
          break;
```

```
case 'D':
case 'd':
        cout << "The grade is 1.0.";
        break;
case 'F':
case 'f':
        cout << "The grade is 0.0.";
        break;
 default:
        cout << "The grade is invalid";
   }
   return 0;
   }
```

If input is 'a' or 'A', output is The grade is 1.0.

If input is 'f', output is The grade is 0.0.

If input is '5', output is The grade is invalid

Forgetting the break;

- No compiler error
- Execution simply "falls thru" other cases until break;

- **Biggest use: MENUs**
  - Provides clearer "big-picture" view
  - Shows menu structure effectively
  - Each branch is one menu choice

switch statement is a natural choice for menu-driven program:

- display the menu
- then, get the user's menu selection
- use user input as expression in switch statement
- use menu choices as expr in case statements

```
|-------------------------------------|
|  MENU FOR BANK TRANSACTIONS         |
|                                     |
|  1  Check balance                   |
|  2  Make a deposit                  |
|  3  Make a withdrawal               |
|  4  Exit                            |
|                                     |
|-------------------------------------|
Enter menu option:
```

```
switch(option)// example using integer values for cases
      case 1:
        cout << "The-account-balance-is:-R-" << balance;
             break;
      case 2:
       cout << "Deposit-amount?-";
       cin >> deposit;
       cout << "Deposit-amount-is-R" << deposit;
       cout << "The-account-balance-is:-R-"
            << balance + deposit << endl;
            balance += deposit;
            break;
      case 3:
         cout << "Withdrawal-amount?-";
         cin >> withdraw;
         cout << "Withdraw-amount-is-R" << withdraw << endl;
         cout << "The-account-balance-is:-R-"
              << balance - withdraw << endl;
              balance -= withdraw;
             break;
```

```
case 4:
    case 4:    cout  << "Thank-you-for-using-this-service.-"<<e
    break

 default:
cout << "Invalid-menu-option!"<< endl; ;
    }
```

Question

Write a C++ program that determines the day of the week based on a numerical input by the user(1 for Monday, 2 for Tuesday, 3 for Wednesday, 4 for Thursday , 5 for Friday, 6 for Saturday and 7 for Sunday).
The program should display an appropriate message indicating the day entered.

Present this solution in two forms:

1. Using if-else statements (5 marks)
1. Using switch statements (5 marks)